SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-002: Forgot Password and Password Reset Feature

Field	Details
Change Request ID	CR-002
Change Request Title	Forgot Password and Password Reset Feature
Student Name	J. Thushany
Student ID	JF24ICT09
Implementation Status	Implemented — Testing Verification Required
Date	15 October 2026

1. Change Request Summary

1.1 Business Requirement

Users who forget their passwords need a secure and convenient way to regain access to their accounts without requiring an administrator to manually change their passwords.

The purpose of CR-002 is to introduce a Forgot Password facility that allows users to request a password reset using their registered email address and set a new password through a time-limited reset token.

1.2 Existing Behaviour

Before implementation:

- Users accessed the system through the existing login page.
- Login required a valid username and password.
- An explicit password recovery facility was not available.
- Users who forgot their passwords had no dedicated password reset workflow.
- Existing authentication and role-based access control were used to protect application routes.

1.3 Requested Change

The following functionality was required:

- Add a Forgot Password link or button to the login interface.

- Allow users to submit their registered email address.
- Generate a cryptographically secure password reset token.
- Store only the token hash in the database.
- Set a token expiration time of 15 minutes.
- Reject invalid, expired, and previously used reset tokens.
- Provide a reset password form with password and confirmation fields.
- Validate that both passwords match.
- Require a minimum password length of eight characters.
- Hash the new password before saving it.
- Return a generic response for unknown email addresses.
- Preserve existing login, authentication, and administrator functionality.

2. Implementation

2.1 Implementation Summary

CR-002 was implemented by extending the existing authentication workflow in the Student Management System.

The frontend provides Forgot Password and Reset Password pages. The backend provides endpoints for requesting a reset token and resetting a password. A new database table, password_reset_tokens, stores token hashes, expiration timestamps, and token usage information.

The implementation uses cryptographically generated random tokens and SHA-256 hashing to avoid storing the original reset token in the database. The new password is hashed using bcrypt before it is saved.

The reset operation uses a database transaction to update the password and invalidate the token together.

Development limitation: An email delivery service has not yet been configured. During development, the reset token is returned by the API to support testing. This behaviour must be removed before production deployment and replaced with secure email delivery.

2.2 Affected Files

File	Change	Purpose
frontend/src/components/LoginFormSection.jsx	Modified	Added navigation from the Forgot Password control to the recovery page.
frontend/src/pages/ForgotPasswordPage.jsx	Added / Verified	Provides the registered email submission interface and reset request feedback.
frontend/src/pages/ResetPasswordPage.jsx	Added / Verified	Provides new password and confirmation fields and handles the reset workflow.
frontend/src/services/authService.js	Added / Verified	Sends Forgot Password and Reset Password API requests.
frontend/src/routes/AppRoutes.jsx	Modified / Verified	Registers the Forgot Password and Reset Password routes.
backend/src/controllers/authController.js	Modified	Implements password recovery and reset request handling while preserving login functionality.
backend/src/models/userModel.js	Modified	Implements email lookup, reset token storage, and transactional password reset operations.
backend/src/routes/authRoutes.js	Verified	Registers the authentication, Forgot Password, and Reset Password endpoints.
database/password_reset_tokens	Added	Stores reset token hashes, expiration times, and usage information.

Note: Confirm the actual Git diff before submitting this table. Keep only files that were genuinely changed or added.

2.3 Technical Changes

Secure Token Generation

A random token is generated using Node.js crypto.randomBytes(32). The token is hashed using SHA-256 before being stored in the database.

Token Expiration

Each reset token is assigned an expiration time of 15 minutes. Expired tokens are rejected during password reset.

Password Validation

The reset endpoint validates the token, checks that the new password and confirmation match, and enforces a minimum password length of eight characters.

Password Hashing

The new password is hashed using bcrypt with a work factor of 12 before updating the user's password_hash field.

Database Transaction

The password update and token consumption are performed in a transaction. This helps prevent a reset token from being successfully reused.

Generic Email Response

The same response message is returned whether or not an account is found for the submitted email address. This reduces the risk of revealing which email addresses are registered.

Existing Authentication Preservation

The existing username and password login flow remains in place. The Last Login update and JWT generation are also preserved.

2.4 Implementation Logic

1. The user selects Forgot Password on the login page.
2. The frontend navigates to the Forgot Password page.
3. The user enters a registered email address.
4. The frontend sends a POST request to /api/auth/forgot-password.
5. The backend searches for the corresponding user account.
6. For a matching account, a secure random reset token is generated.
7. The SHA-256 token hash and expiration timestamp are stored in password_reset_tokens.

8. During development, the original token is returned in the API response for testing.
9. The user opens the reset page using the supplied token.
10. The user enters and confirms a new password.
11. The frontend validates the password fields and submits a POST request to /api/auth/reset-password.
12. The backend hashes the submitted token and validates it against the database.
13. The backend checks the password requirements and hashes the new password using bcrypt.
14. The password is updated and the token is marked as used within a database transaction.
15. The user can return to the login page and attempt to sign in with the new password.

2.5 Error and Data Handling

The implementation handles missing email addresses, missing reset fields, mismatched passwords, passwords shorter than eight characters, and invalid or expired reset tokens.

Unknown email addresses receive a generic response rather than an explicit account-not-found message.

Database or server errors are logged on the backend, while the API returns a general internal server error response.

The reset token is stored as a hash in the database, and the password is stored as a bcrypt hash.

3. Testing and Verification

3.1 Testing Approach

The implementation is intended to be verified through manual frontend and backend testing.

Testing types:

- Functional testing
- Positive testing
- Input validation testing
- Token expiration and reuse testing
- Database verification

- UI testing
- Regression testing
- Authentication verification

Environment: Development

Browser: Google Chrome

Test Role: Student / existing user

Important: Backend and frontend startup have been confirmed. Individual test results below must be marked Pass only after each corresponding test has actually been performed.

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Open Forgot Password from the login page	Forgot Password page opens	To be recorded	Pending
TC-02	Submit a registered student email	Generic reset response is returned and a development reset token is available	To be recorded	Pending
TC-03	Submit an unknown email address	Generic response is returned without revealing account existence	To be recorded	Pending
TC-04	Submit mismatched passwords	Password reset is rejected	To be recorded	Pending
TC-05	Submit a password shorter than eight characters	Password validation error is displayed	To be recorded	Pending
TC-06	Submit a valid, unexpired token with matching passwords	Password is reset successfully	To be recorded	Pending
TC-07	Reuse a consumed reset token	Reset request is rejected	To be recorded	Pending

TC-08	Attempt to reset with an invalid or expired token	Reset request is rejected	To be recorded	Pending
TC-09	Log in using the new password	Login succeeds with the new password	To be recorded	Pending
TC-10	Verify existing administrator login and dashboard	Existing authentication functionality continues to work	To be recorded	Pending

3.3 Test Summary

Test Result	**Count**
Total Test Cases	10
Passed	To be recorded
Failed	To be recorded
Pending	10
Pass Rate	To be calculated after testing

Overall Test Result: Pending full functional verification.

4. Acceptance Criteria Verification

ID	**Acceptance Criterion**	**Verification Method**	**Status**
AC-001	Forgot Password is accessible from the login page.	Frontend navigation test	Implemented / Verify
AC-002	Users can request a reset using their registered email address.	API and database test	Implemented / Verify
AC-003	Reset tokens are securely generated and stored as hashes.	Code and database inspection	Implemented
AC-004	Reset tokens expire after 15 minutes.	Expiration validation test	Implemented / Verify
AC-005	Invalid, expired, and reused tokens are rejected.	Negative testing	Implemented / Verify

AC-006	New password and confirmation must match.	Input validation test	Implemented / Verify
AC-007	Passwords must contain at least eight characters.	Password validation test	Implemented
AC-008	New passwords are hashed before database storage.	Code and database inspection	Implemented
AC-009	Unknown email addresses receive a generic response.	API response test	Implemented / Verify
AC-010	Existing login and administrator functionality remain operational.	Regression testing	Implemented / Verify
AC-011	Password reset instructions are delivered through email.	Email delivery test	Pending — email service not configured

5. Evidence

Attach a maximum of two relevant screenshots.

Evidence 01 — Forgot Password Interface

Related criteria: AC-001, AC-002, AC-009.

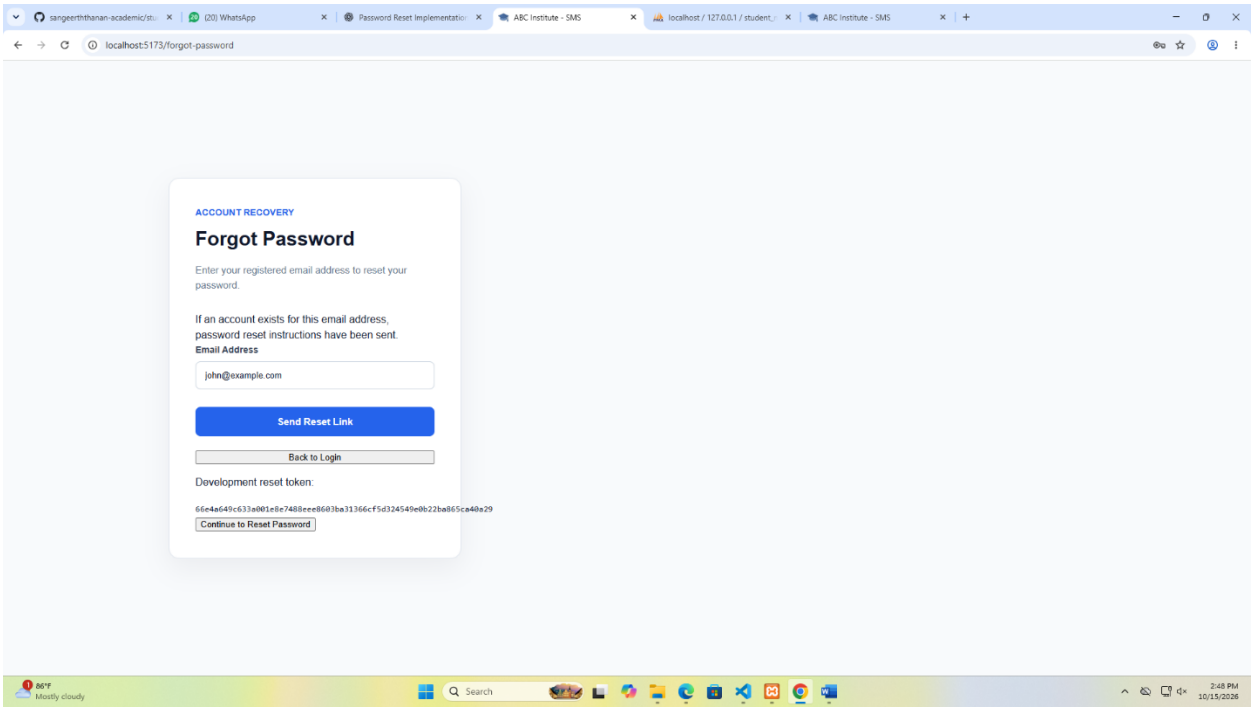


Figure 1: Forgot Password page displaying the registered email input, submission control, and request feedback.

Evidence 02 — Reset Password Interface

Related criteria: AC-004, AC-005, AC-006, AC-007.

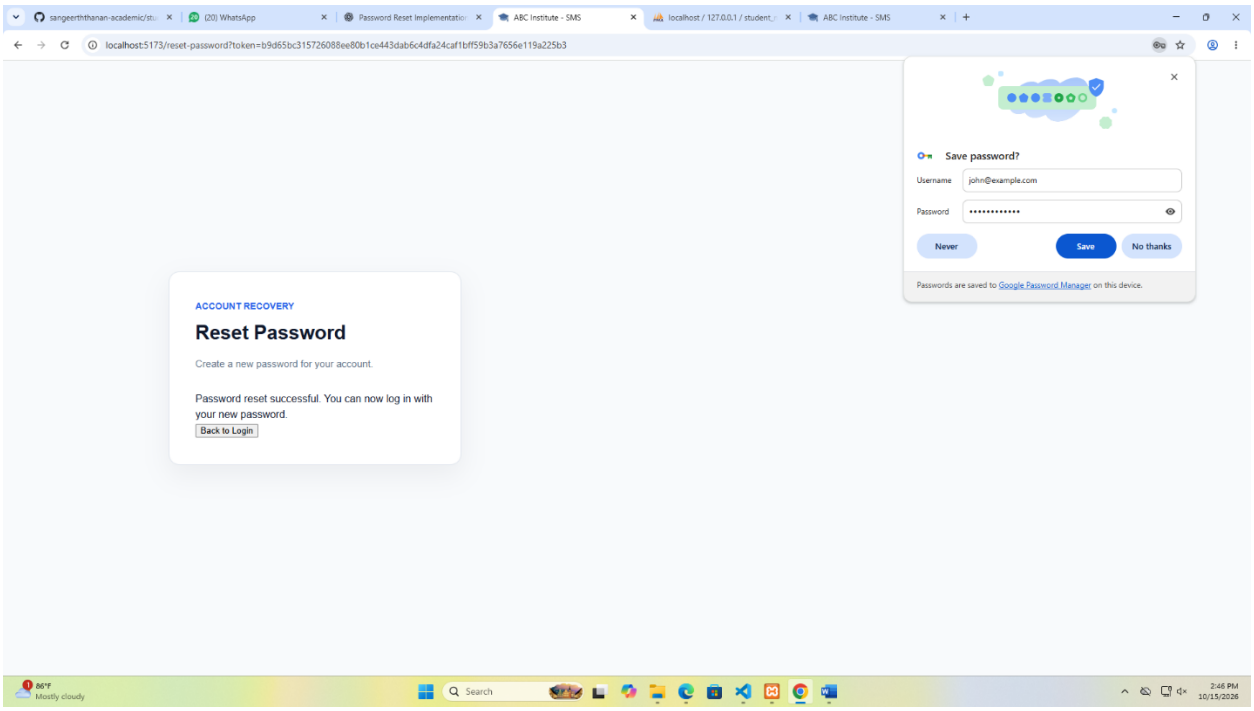


Figure 2: Reset Password page displaying the new password and confirmation fields, including a validation message or successful reset result.

Additional evidence, if required:

- Database screenshot showing the password_reset_tokens table and token hash.
- Browser screenshot showing the successful login using the new password.
- Git branch, commit, and push evidence.

Do not include real passwords or active reset tokens in submitted screenshots.

6. Final Result

Item	Result
Requirements	Implemented with email-delivery limitation
Implementation	Core development implementation completed

Testing	Pending full functional verification
Acceptance Criteria	Partially verified
Existing Authentication	Backend and frontend startup confirmed; regression tests pending
Final Status	Pending testing and production email configuration

6.1 Final Summary

CR-002 introduces a Forgot Password and Password Reset workflow to the Student Management System. The implementation adds frontend recovery pages, backend authentication endpoints, and a database table for storing reset token hashes and expiration information.

The reset process validates the submitted token, enforces password matching and minimum-length requirements, hashes the new password using bcrypt, and consumes the token as part of a database transaction. A generic response is used for unknown email addresses to reduce account enumeration.

The existing login and Last Login tracking logic have been preserved in the updated authentication controller.

The backend and frontend have been confirmed to start successfully. Full functional verification is still required before declaring all acceptance criteria passed. Email delivery must also be configured before the feature is deployed to production.

6.2 Issues and Limitations

1. **Email delivery:** An email provider has not yet been configured. The reset token is currently returned by the API for development testing only.
2. **Production security:** The development token response must be removed when secure email delivery is implemented.
3. **Testing:** All positive, negative, token expiry, token reuse, and regression test cases must be completed and recorded before final acceptance.
4. **Email association:** The current lookup uses the email associated with a student record. Administrator account recovery requires a registered email association or a separate approved recovery process.

Student Name: J. Thushany

Student ID: JF24ICT09

Date: 15 October 2026

Signature: _____

~ END OF REPORT ~